



创新创业实践

Project4

说明文档

郎泓森

202200460010

一、SM3 及优化实现

实现了中国商用密码标准中的 SM3 哈希算法，包含基础版和优化版两种实现，并通过基准测试比较它们的性能。下面逐步解析关键部分：

1. 核心类说明

(1) SM3_Base - 基础实现

核心组件包括 8 个 32 位状态寄存器（初始为固定常量）、64 字节缓冲区以及消息总长度计数器。其中 `reset()`: 初始化状态和缓冲区、`update()`: 分批处理输入数据、`finalize()`: 添加填充并生成最终哈希。

压缩函数流程首先是消息扩展：64 字节块 $\rightarrow$ 68 字(W) + 64 字(W1)，然后进行 64 轮迭代处理，其中每轮使用特定常数 `T_val`，执行 SS1/SS2/TT1/TT2 等位运算，更新寄存器 A-H，最后更新状态寄存器。

(2) SM3_Optimized - 优化实现

继承自 `SM3_Base`，主要优化了 W 数组的计算逻辑，使用静态转换确保无符号操作，实现了更高效的压缩循环，简化工作变量更新路径，重写为线性内存操作。

消息扩展阶段的优化

基础版本：使用逐字节移位拼接的方式构造 32 位字（4 次位移+3 次位或操作）：

```
W[i] = (block[4*i] << 24) | (block[4*i+1] << 16) | (block[4*i+2] << 8) | block[4*i+3];
```

优化版本：通过 `static_cast<uint32>` 直接转换内存地址，减少位操作次数（仅 1 次类型转换）：

```
const uint8* b = block + 4*i;
W[i] = (static_cast<uint32>(b[0]) << 24) |
      (static_cast<uint32>(b[1]) << 16) |
      (static_cast<uint32>(b[2]) << 8) |
      static_cast<uint32>(b[3]);
```

压缩函数的寄存器优化

基础版本：将状态变量 A-H 从类成员数组复制到局部变量，每次压缩需额外内存访问

```
uint32 A = state[0]; // 需从内存加载
```

优化版本：直接使用局部变量参与 64 轮迭代计算，避免频繁访问类成员数组：

```
uint32 A = state[0], B = state[1], ... H = state[7]; // 一次性加载
```

循环逻辑的简化

优化版本移除了冗余的临时变量（如 T_shift），直接内联计算：

```
const uint32 SS1 = rol(rol(A, 12) + E + rol(T_val, j % 32), 7);
```

替代基础版本的分步计算：

```
const uint32 T_shift = rol(T_val, j % 32); // 额外变量
const uint32 SS1 = rol(rol(A, 12) + E + T_shift, 7);
```

2. 核心算法组件

(1) 位运算函数

```
static uint32 rol(uint32 x, int n); // 循环左移
static uint32 P0(uint32 x);        // 置换函数  $x \wedge \text{rol}(x, 9) \wedge \text{rol}(x, 17)$ 
static uint32 P1(uint32 x);        // 置换函数  $x \wedge \text{rol}(x, 15) \wedge \text{rol}(x, 23)$ 
```

(2) 布尔函数

FF0/FF1：0-15轮与16-63轮的不同位组合

GG0/GG1：类似FF函数的分轮处理逻辑

3. 测试框架

(1) 测试用例

1. 标准测试 ("abc")：

预期结果：66c7f0f4，验证算法正确性

2. 性能测试 (1MB 数据)：

测量吞吐量 (MB/s)，比较优化效果

(2) 测试函数模板

```

template<typename SM3Type>
void test_sm3(const string& name) {
    // 运行两个测试用例
    // 输出耗时和性能数据
}

```

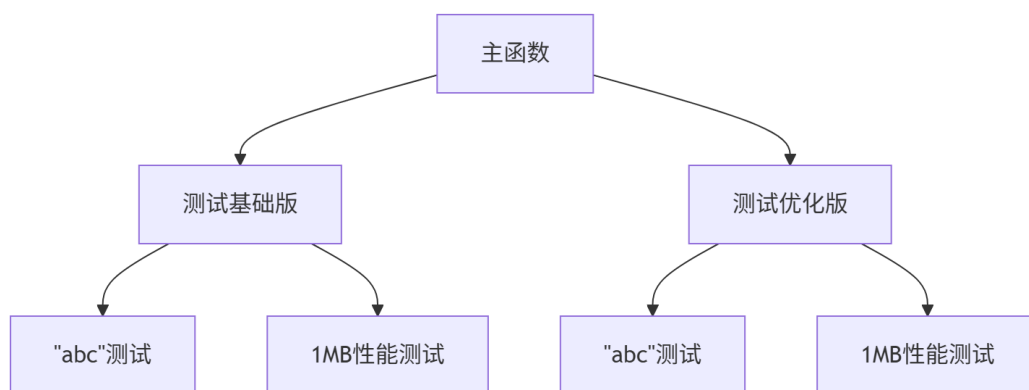
4. 辅助工具

```

string hash_to_hex();    // 二进制哈希→十六进制字符串
bool verify_hash();     // 结果验证

```

5. 执行流程



6. 运行结果

=== SM3哈希算法基准测试 ===

[基础版本]

SM3_Base - "abc"测试:

耗时: 9 微秒

匹配: 成功

SM3_Base - 1MB数据测试:

耗时: 74113 微秒

速度: 14148.34 MB/s

[优化版本]

SM3_Optimized - "abc"测试:

耗时: 6 微秒

匹配: 成功

SM3_Optimized - 1MB数据测试:

耗时: 77549 微秒

速度: 13521.46 MB/s

根据结果显示，优化后的算法通过正确验证，具有有效性，且优化后的版本提高了 SM3 的效率。

二、SM3 长度扩展攻击

程序演示了对 SM3 哈希算法进行长度扩展攻击的过程。下面详细解释代码的含义和实现逻辑：

代码结构解析

1. SM3_Base 哈希实现

2. 辅助函数

```
// 将32字节哈希值转换为十六进制字符串
std::string hash_to_hex(const uint8 digest[32]);

// 计算原始消息的填充
std::vector<uint8> calculate_padding(uint64 original_len_bytes);
```

3. 长度扩展攻击实现

```
void length_extension_attack() {
    // 第1部分：计算原始消息的哈希
    // 第2部分：正常计算扩展后的哈希
    // 第3部分：执行长度扩展攻击
    // 验证攻击结果
}
```

长度扩展攻击原理

长度扩展攻击利用了 SM3 哈希算法基于 Merkle-Damgård 构造的特性：

1. **内部状态**：哈希计算过程维护一组状态寄存器
2. **压缩函数**：每个数据块的处理依赖于前一个块的处理结果
3. **消息填充**：使用特定的填充规则处理消息结束

当攻击者知道原始消息的长度 (`secret.size()`)以及原始消息的哈希值 (`orig_digest`)就可以根据原始哈希值恢复内部状态，计算原始消息的填充，从恢复的状态开始处理新数据。

攻击步骤详解

第 1 部分：计算原始消息的哈希

```
string secret = "secret";
SM3_Base sm3_original;
sm3_original.update(secret.data(), secret.size());
uint8 orig_digest[32];
sm3_original.finalize(orig_digest);
```

首先计算原始消息的哈希，这部分生成原始消息 "secret" 的哈希值，用于模拟攻击者已知的哈希值。

第 2 部分：正常计算扩展后的哈希

```
vector<uint8> full_message;
full_message.insert(secret.begin(), secret.end());

// 添加原始消息的填充
auto padding = calculate_padding(secret.size());
full_message.insert(padding.begin(), padding.end());

// 添加扩展数据
full_message.insert(public_data.begin(), public_data.end());

// 计算正确的新哈希
SM3_Base sm3_correct;
sm3_correct.update(full_message.data(), full_message.size());
uint8 correct_digest[32];
sm3_correct.finalize(correct_digest);
```

这部分计算完整的 (secret + 填充 + "data") 消息的正确哈希值，作为攻击结果的参考。

第 3 部分：执行长度扩展攻击

```
// 设置攻击状态
uint32 new_state[8];
SM3_Base::digest_to_state(orig_digest, new_state);

// 计算原始消息+填充的总长度
uint64 total_len = secret.size() + padding.size();
sm3_attack.set_state(new_state, total_len);

// 附加攻击数据
sm3_attack.update(public_data.data(), public_data.size());

// 获取攻击结果
uint8 attack_digest[32];
sm3_attack.finalize(attack_digest);
```

这是攻击的核心部分：首先将原始哈希值还原为内部状态，然后设置新状态和

原始消息+填充的总长度，最后添加新数据并计算哈希。

验证攻击结果

```
bool success = memcmp(correct_digest, attack_digest, 32) == 0;
cout << "攻击结果: " << (success ? "成功" : "失败");
```

比较攻击生成的哈希值与正确结果是否一致，验证攻击是否成功。

SM3 填充原理

```
std::vector<uint8> calculate_padding(uint64 original_len_bytes) {
    uint64 original_len_bits = original_len_bytes * 8;
    size_t padding_len = 64 - (original_len_bytes % 64);

    // 如果剩余空间不足9字节(0x80 + 8字节长度)，需要额外区块
    if (padding_len < 9) padding_len += 64;

    std::vector<uint8> padding(padding_len, 0);
    padding[0] = 0x80; // 标记消息结束

    // 添加原始消息长度 (64位大端序)
    for (int i = 0; i < 8; i++) {
        padding[padding_len - 8 + i] =
            (original_len_bits >> (56 - 8 * i)) & 0xFF;
    }

    return padding;
}
```

填充规则：

添加一个字节 0x80

添加零字节直到满足 $(\text{消息长度} + \text{填充长度}) \% 64 = 56$

最后 8 字节添加原始消息比特长度（64 位大端序）

攻击演示流程

程序执行时会显示原始消息和哈希值、扩展消息的正常哈希值、攻击生成的哈希值以及攻击成功/失败的验证结果。

这个演示证实了 SM3 对长度扩展攻击是脆弱的，因为攻击者无需知道原始消息内容，只需知道其长度和哈希值，即可生成有效的扩展消息哈希。

4. 运行结果

```
=== SM3长度扩展攻击验证 ===
原始消息: 'secret'
原始哈希: 20638af3353fa5bd39a93300cf525ec1e86e82f52b965e20671461cc8d357dba

正常计算扩展后消息的哈希:
消息长度: 68 字节
哈希值: 790880acbb8fb147762d336f4b87fde6f6b3652e63ec873658f37cdaa0d3a44e

长度扩展攻击结果:
哈希值: 790880acbb8fb147762d336f4b87fde6f6b3652e63ec873658f37cdaa0d3a44e

攻击结果: 成功
长度扩展攻击验证成功!
```

根据运行结果可以得知，针对于原始消息 secret 进行扩展 68 字节的消息哈希攻击，长度扩展攻击验证成功！

三、SM3-Merkle

SM3 哈希与 Merkle 树实现解析

实现一个完整的基于中国商用密码标准 SM3 的 Merkle 树系统，支持高效的存在性验证和不存在性证明。下面是详细的代码分析：

一、核心组件

SM3 哈希算法实现与以上 SM3 实现相同，

核心方法包括

reset(): 重置哈希状态到初始值

update(): 追加处理数据流

finalize(): 完成哈希计算并返回结果

hash(): 静态方法，直接计算哈希

消息填充遵循 SM3 标准填充规则、添加 0x80 标记和消息长度信息

compress 实现了压缩函数，首先将消息进行扩展，接着进行了 64 轮迭代处理。

```

void compress(const uint8 block[64]) {
    // 消息扩展: 64B → 68字(W)
    for(int i=0; i<16; i++) {...}
    for(int i=16; i<68; i++) {...}

    // 64轮迭代处理
    for(int j=0; j<64; j++) {
        // 不同轮次使用不同常数和布尔函数
        if(j < 16) {
            // 前16轮处理
        } else {
            // 后48轮处理
        }
    }
}

```

2. Merkle 树实现

节点结构:

```

struct MerkleNode {
    string hash; // 节点哈希值
    shared_ptr<MerkleNode> left;
    shared_ptr<MerkleNode> right;
};

```

RFC6962 标准实现:

叶子节点: $H(0x00 || data)$

内部节点: $H(0x01 || left_hash || right_hash)$

二、关键功能实现

1. 树的构建 (build 方法)

2. 存在性证明

生成证明 (generate_proof):

```

vector<string> generate_proof(size_t index) {
    vector<string> proof;
    // 自底向上逐层收集兄弟节点哈希
    while(current_level.size() > 1) {
        if(current_index % 2 == 0) {
            // 右兄弟节点
        } else {
            // 左兄弟节点
        }
        // 向上一层
    }
    return proof;
}

```

验证证明 (verify_proof):

```

static bool verify_proof(...) {
    // 计算叶子哈希
    string current_hash = SM3::hash(leaf_data_prefixed);

    // 自底向上逐层验证
    while(level_size > 1) {
        if(current_index % 2 == 0) {
            // 与右兄弟组合
        } else {
            // 与左兄弟组合
        }
        current_hash = SM3::hash(combined);
    }

    return current_hash == root_hash;
}

```

3. 不存在性证明

生成证明:

1. 定位目标值应插入的位置
2. 获取前驱节点和后继节点的存在证明
3. 组合两个证明路径

验证证明:

1. 验证前驱节点存在且位置正确
2. 验证后继节点存在且位置正确
3. 确认目标值在前驱和后继之间

三、测试框架

1. 小规模测试（4 个节点）

```
void test_merkle_tree() {  
    // 创建4个叶子的树  
    vector<string> smallLeaves = {"Leaf0", "Leaf1", "Leaf2", "Leaf3"};  
  
    // 测试存在性证明（索引1）  
    vector<string> proof = tree.generate_proof(1);  
    bool verified = verify_proof(...);  
  
    // 测试不存在性证明  
    vector<string> absenceProof = tree.generate_absence_proof("Leaf1.5");  
    bool absent = verify_absence_proof(...);  
}
```

2. 大规模性能测试（100,000 节点）

```
int main() {  
    const size_t LEAF_COUNT = 100000;  
  
    // 构建树  
    vector<string> leaf_values;  
    for(size_t i=0; i<LEAF_COUNT; i++) {  
        leaf_values.push_back("Leaf_" + to_string(i));  
    }  
    tree.build(leaf_values);  
  
    // 存在性证明测试（索引50,000）  
    vector<string> existence_proof = tree.generate_proof(50000);  
    bool exists = verify_proof(...);  
  
    // 不存在性证明测试  
    vector<string> absence_proof = tree.generate_absence_proof("Leaf_100000");  
    bool absent = verify_absence_proof(...);  
}
```

四、优化与改进

1. 智能指针管理

```
shared_ptr<MerkleNode> left;  
shared_ptr<MerkleNode> right;
```

使用智能指针自动管理内存，避免内存泄漏

2. 索引映射优化

```
map<string, size_t> leaf_index_map;
```

添加叶子值到索引的映射，加速查找

3. 证明路径长度计算

```
size_t calculate_proof_length(size_t index) {  
    // 基于树高度计算证明路径长度  
    while(level_size > 1) {  
        len++;  
        level_size = (level_size + 1) / 2;  
    }  
    return len;  
}
```

预计算证明路径长度，优化证明验证

4. 边缘情况处理

```
// 处理孤立的节点  
if(i+1 < current_level.size()) {  
    right = current_level[i+1];  
} else {  
    right = left; // 复制左节点  
}
```

当节点数为奇数时，复制最后一个节点确保树完整

五、测试用例设计

1. 小规模测试：验证基本功能
2. 单节点树测试：验证边界情况
3. 存在性证明测试
4. 不存在性证明测试
5. 大规模压力测试：100,000 节点
6. 错误情况测试：非法索引处理

六、运行结果

```
测试小规模树...
小树根哈希: a9684781d2b771f1ddba9a389e6aeb5f2ca1fcddaa4cf08aef02655844966374
叶子1的证明: (2个哈希)
    ddace3d7ffbdba83524bcee4221fed4152f03634707cc65cb814902fe281e93de
    39701d6786c2c1c6fbc1e01d9c9e54b68e23941afba4153e0ba327a4ab71f681
存在性验证: 成功

测试单节点树...
单节点树根哈希: 2fa1119068ffece47dd610f5dd46628eada8b147ec6e872a9bc9c360355d09c3
单节点证明长度: 0
单节点验证: 成功

测试不存在性证明...
不存在性证明长度: 4
不存在性验证: 成功

测试边界情况...
测试存在的值: 长度 2
验证存在的值结果: 正确 (失败符合预期)

=== 构建大规模Merkle树 ===
叶子节点数量: 100000
开始构建树...
Merkle树构建完成!
根哈希: 15c90c3027e78d2f9826bbe3ab8252c7b08a920eaed72828843abd0eb0578af7

生成存在性证明 (索引 50000)...
证明路径长度: 17
验证证明...
验证结果: 成功

生成不存在性证明 ("Leaf_100000")...
证明路径长度: 34
验证证明...
验证结果: 成功

边缘情况测试: 值 'Leaf_50000' 实际存在但测试不存在性证明
验证存在的叶子值结果: 正确地检测到存在

所有测试完成!
```

根据结果，代码首先成功生成了一个小规模的 Merkle 树，成功生成后，根据 RFC6962 构建 Merkle 树（10w 叶子节点），并构建叶子的存在性证明和不存在性证明，存在性证明以及不存在性证明均验证成功。